CloudFront Origin Group – Geographical Failover with Load Balancers

To Begin with the Lab

Summary of the Lab

In this lab, Amazon CloudFront Origin Groups were configured to provide automatic geographical failover between Application Load Balancers deployed in Mumbai and N. Virginia. The Mumbai load balancer acted as the primary origin while the N. Virginia load balancer served as the backup origin. A CloudFront distribution was created and configured with both origins inside an origin group, allowing CloudFront to switch traffic automatically when the primary origin became unavailable. The failover process was tested by disabling HTTP access on the Mumbai load balancer and verifying that CloudFront redirected requests to the US load balancer. Finally, the primary origin was restored and CloudFront resumed serving traffic from Mumbai.

Understanding Geographical Failover

Geographical failover is a disaster recovery strategy where workloads are duplicated across multiple AWS Regions. If one region experiences an outage, traffic is automatically redirected to infrastructure running in another region. This ensures users continue accessing the application even during regional failures.

Example:

An e-commerce application serves customers from Mumbai under normal conditions. During a regional outage affecting ap-south-1, CloudFront routes requests to a backup environment hosted in us-east-1, allowing customers to continue browsing and purchasing products.

Understanding Failover Testing

Failover testing validates that CloudFront correctly switches traffic to the backup origin when the primary origin becomes unavailable. This ensures disaster recovery mechanisms function as expected before production deployment.

Example:

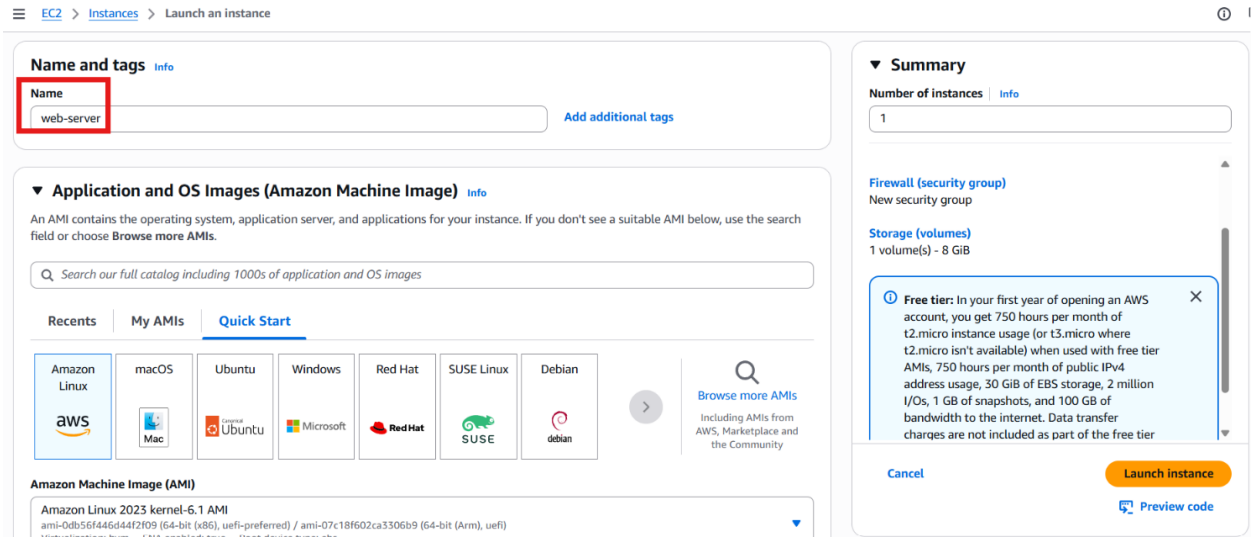
Removing inbound HTTP access from the Mumbai load balancer prevents it from receiving requests. CloudFront detects the failure and starts serving traffic from the Virginia load balancer.

Lab Steps

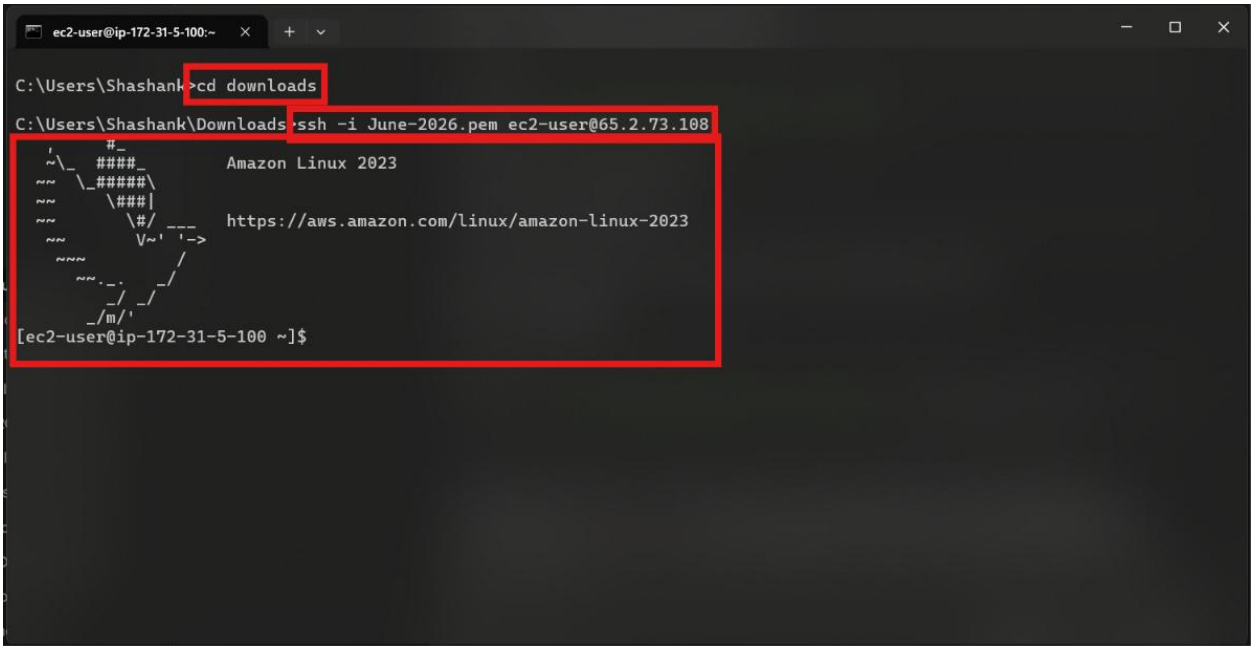
1. Prepare Regional Web Servers

- Sign in to the AWS Management Console.
- Launch two EC2 web servers in **Mumbai (ap-south-1)**.

- Launch two EC2 web servers in **N. Virginia (us-east-1)**.
- Follow these steps for creating the respective instances.
- Create an **Amazon EC2** instance and configure name as **web-server-1** and enable **Allow HTTP traffic from the internet.**



- Now we will configure it as a web server so we first have to access the instance.
- For this, we will use **Command Prompt** and run the following commands.
- Now type: **cd {location_of_your_security_key}.**
- Now type: **ssh -i{security_key_name} ec2-user@{public_ipv4_address_of_ec2_instance}.**



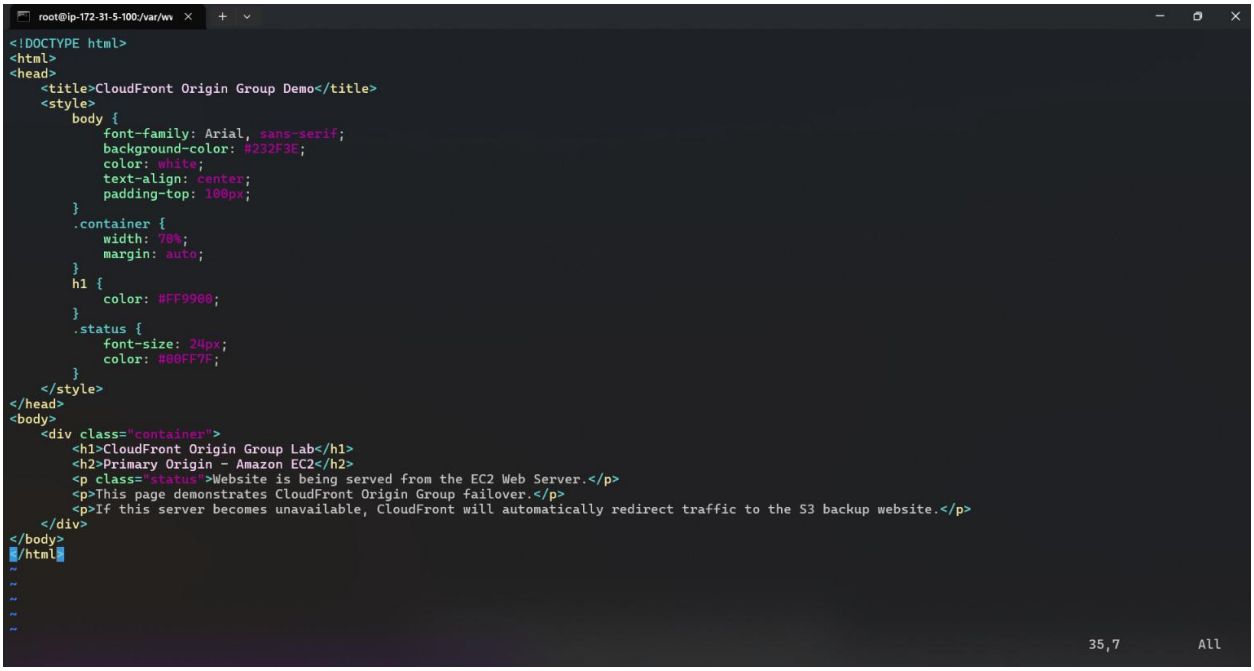
- Now login as root user by typing this command: **sudo -i.**
- Use **yum install httpd** command to configure and place webpage in Apache on the EC2 instance.

```
root@ip-172-31-6-87:~
[ec2-user@ip-172-31-6-87 ~]$ sudo -i
[root@ip-172-31-6-87 ~]# yum install httpd
Amazon Linux 2023 Kernel Livepatch repository                    392 kB/s |  47 kB     00:00
Last metadata expiration check: 0:00:01 ago on Fri Jun 12 09:07:49 2026.
Dependencies resolved.
================================================================================
 Package                 Architecture  Version                    Repository      Size
================================================================================
Installing:
 httpd                   x86_64        2.4.67-1.amzn2023.0.1      amazonlinux     46 k
Installing dependencies:
 apr                     x86_64        1.7.5-1.amzn2023.0.4       amazonlinux    129 k
 apr-util                x86_64        1.6.3-1.amzn2023.0.2       amazonlinux     97 k
 apr-util-lmdb           x86_64        1.6.3-1.amzn2023.0.2       amazonlinux     13 k
 generic-logos-httpd     noarch        18.0.0-12.amzn2023.0.3     amazonlinux     19 k
 httpd-core              x86_64        2.4.67-1.amzn2023.0.1      amazonlinux    1.4 M
 httpd-filesystem        noarch        2.4.67-1.amzn2023.0.1      amazonlinux     12 k
 httpd-tools             x86_64        2.4.67-1.amzn2023.0.1      amazonlinux     80 k
 libbrotli               x86_64        1.0.9-4.amzn2023.0.2       amazonlinux    315 k
 mailcap                 noarch        2.1.49-3.amzn2023.0.3      amazonlinux     33 k
Installing weak dependencies:
 apr-util-openssl        x86_64        1.6.3-1.amzn2023.0.2       amazonlinux     15 k
 mod_http2               x86_64        2.0.39-1.amzn2023.0.1      amazonlinux    167 k
 mod_lua                 x86_64        2.4.67-1.amzn2023.0.1      amazonlinux     59 k

Transaction Summary
================================================================================
Install  13 Packages

Total download size: 2.4 M
```

- Replace the default web page with the provided index.html content with the help of vi editor.

```
Installed:
  apr-1.7.5-1.amzn2023.0.4.x86_64                apr-util-1.6.3-1.amzn2023.0.2.x86_64
  apr-util-lmdb-1.6.3-1.amzn2023.0.2.x86_64      apr-util-openssl-1.6.3-1.amzn2023.0.2.x86_64
  generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch  httpd-2.4.67-1.amzn2023.0.1.x86_64
  httpd-core-2.4.67-1.amzn2023.0.1.x86_64        httpd-filesystem-2.4.67-1.amzn2023.0.1.noarch
  httpd-tools-2.4.67-1.amzn2023.0.1.x86_64       libbrotli-1.0.9-4.amzn2023.0.2.x86_64
  mailcap-2.1.49-3.amzn2023.0.3.noarch           mod_http2-2.0.39-1.amzn2023.0.1.x86_64
  mod_lua-2.4.67-1.amzn2023.0.1.x86_64

Complete!
[root@ip-172-31-5-100 ~]# cd /var/www/html/
[root@ip-172-31-5-100 html]# vi index.html
```

- Use the above commands to change content of the default index.html and paste your code like this or use the code below.

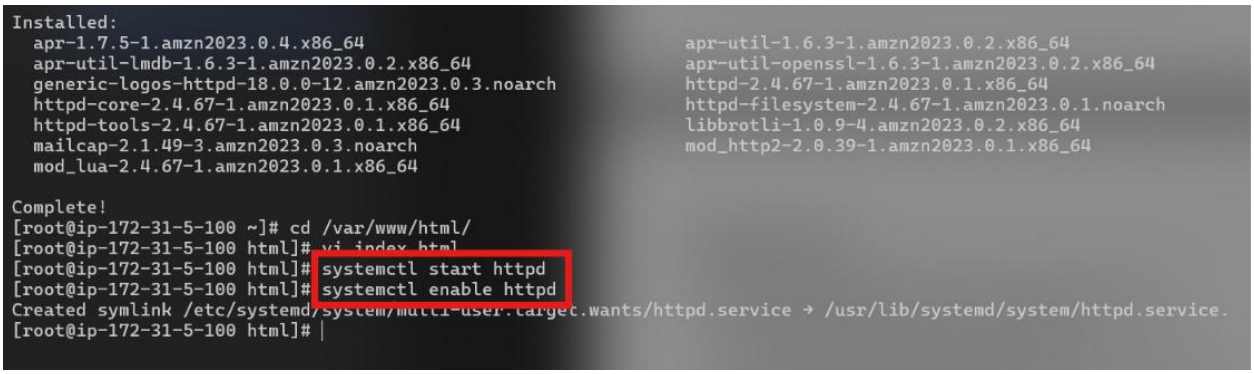



```
<!DOCTYPE html>
<html>
<head>
   <title>CloudFront Origin Group Demo</title>
   <style>
      body {
         font-family: Arial, sans-serif;
         background-color: #232F3E;
         color: white;
         text-align: center;
         padding-top: 100px;
      }
      .container {
         width: 70%;
         margin: auto;
      }
      h1 {
         color: #FF9900;
      }
      .status {
         font-size: 24px;
         color: #00FF7F;
```

```
    }
  </style>
</head>
<body>
  <div class="container">
    <h1>CloudFront Origin Group Lab</h1>
    <h2>Amazon EC2 </h2>
    <p class="status">Website is being served from the EC2 Web Server.</p>
  </div>
</body>
</html>
```

- Now enable and run the web server services using these commands

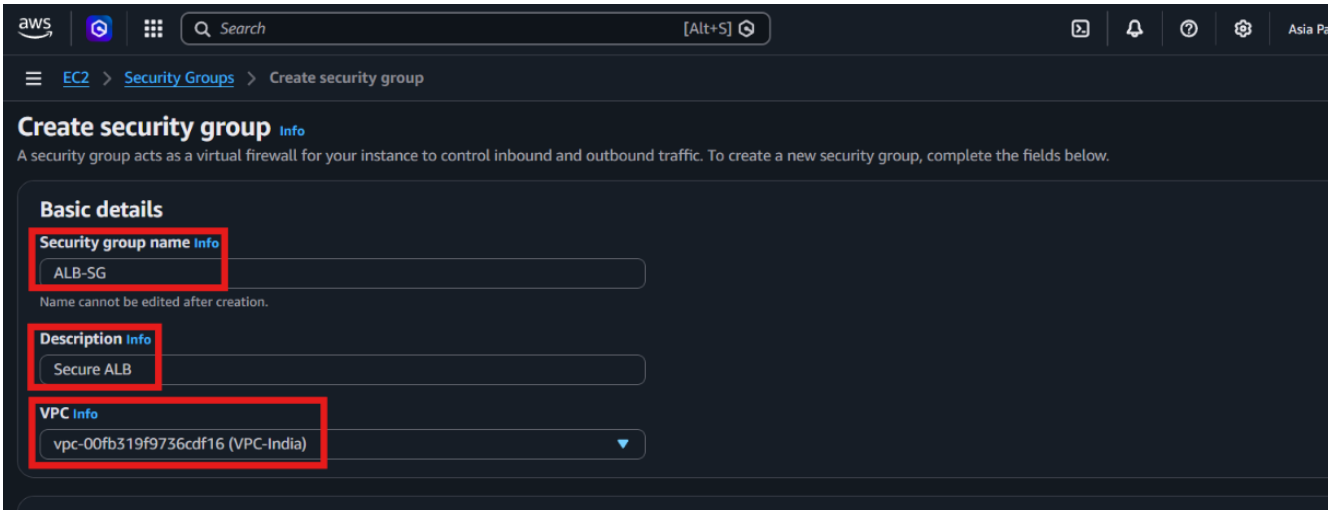
```
systemctl start httpd
systemctl enable httpd
```



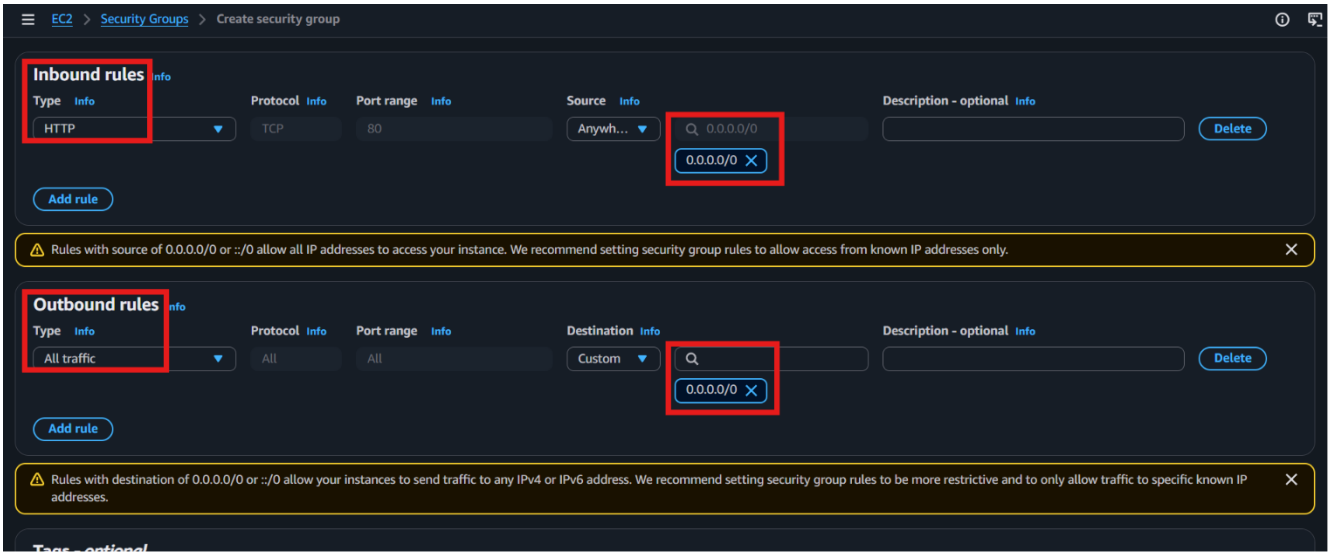
- Verify that the EC2 website opens correctly using the public IP.
- Verify all web servers are accessible.

2. Create Security Groups

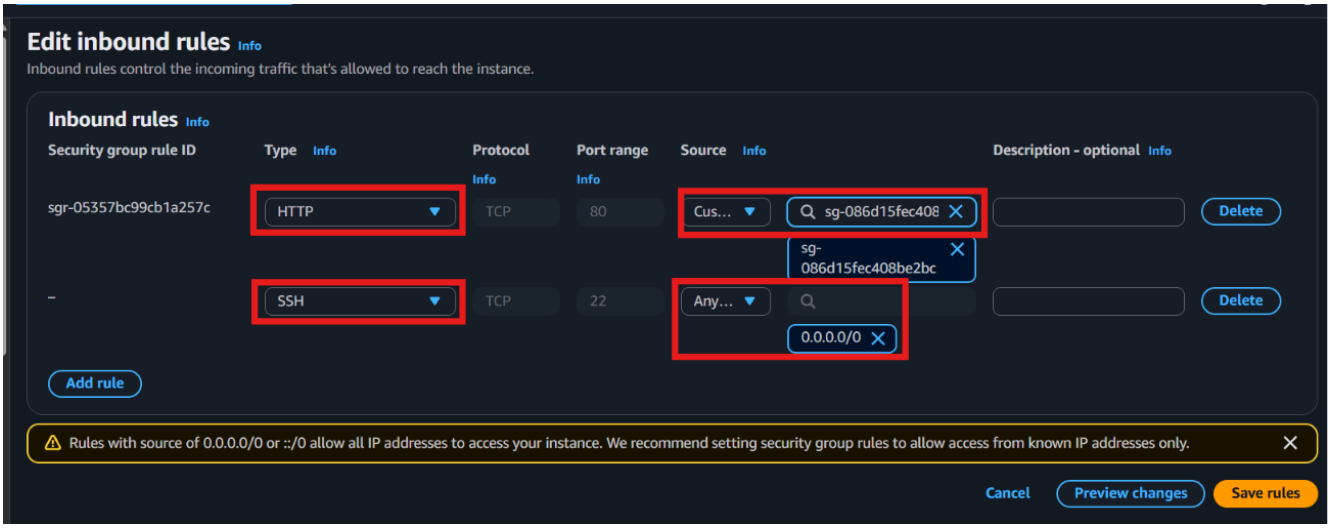
- Click on **EC2 → Security Groups → Create security group.**
- In both regions, create an **ALB-SG** security group that allows HTTP traffic from anywhere on port 80, so configure the name, description and choose default VPC.



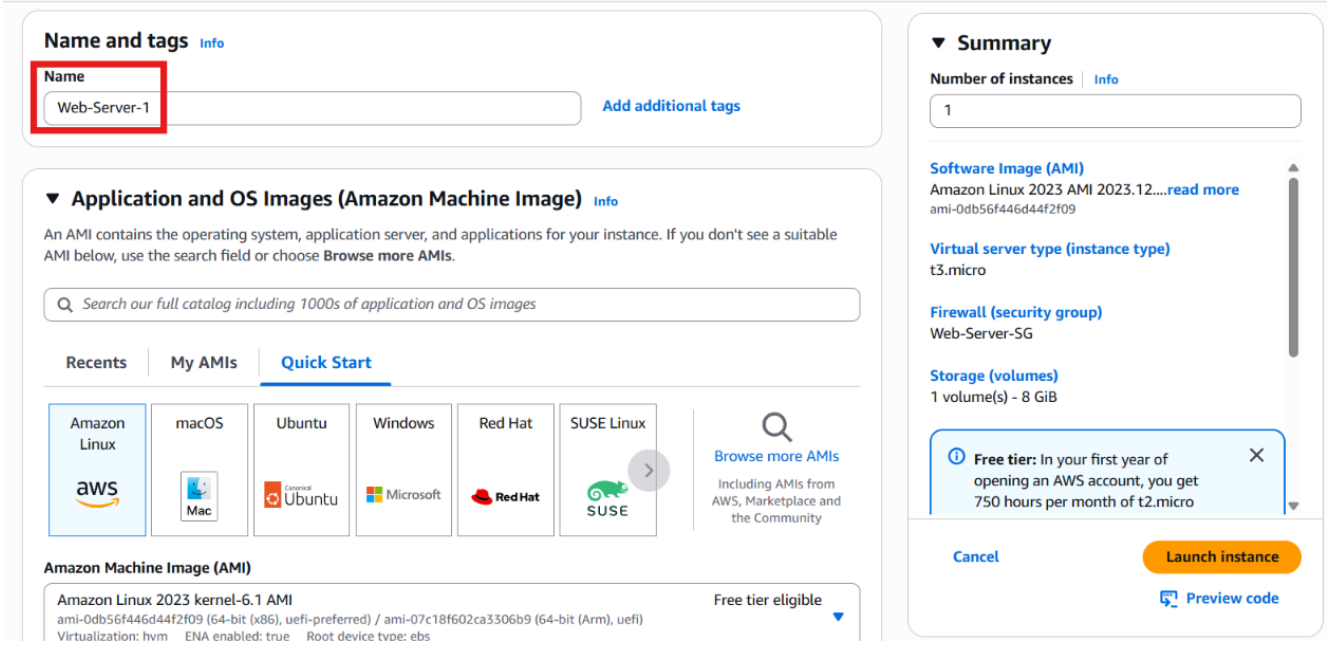
- Choose proper **Inbound and Outbound rule** as shown and click on **Create security group.**



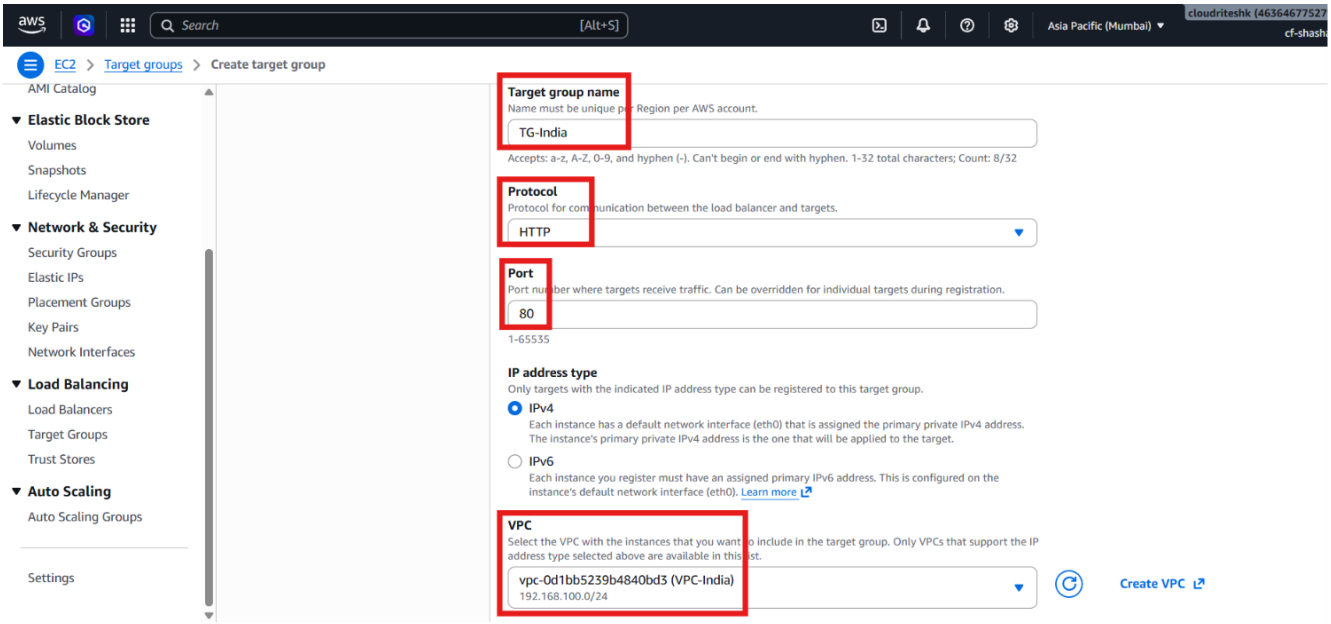
- In both regions, create a **Web-Server-SG** security group that allows HTTP traffic only from the ALB security group with **Inbound and Outbound Rules** as shown.
- Select **ALB-SG** as the **Source** in **Inbound Rule** while creating Web-Server-SG.
- Add an SSH inbound rule to **WebServer-SG** from 0.0.0.0/0 so the instances can be managed only if needed.



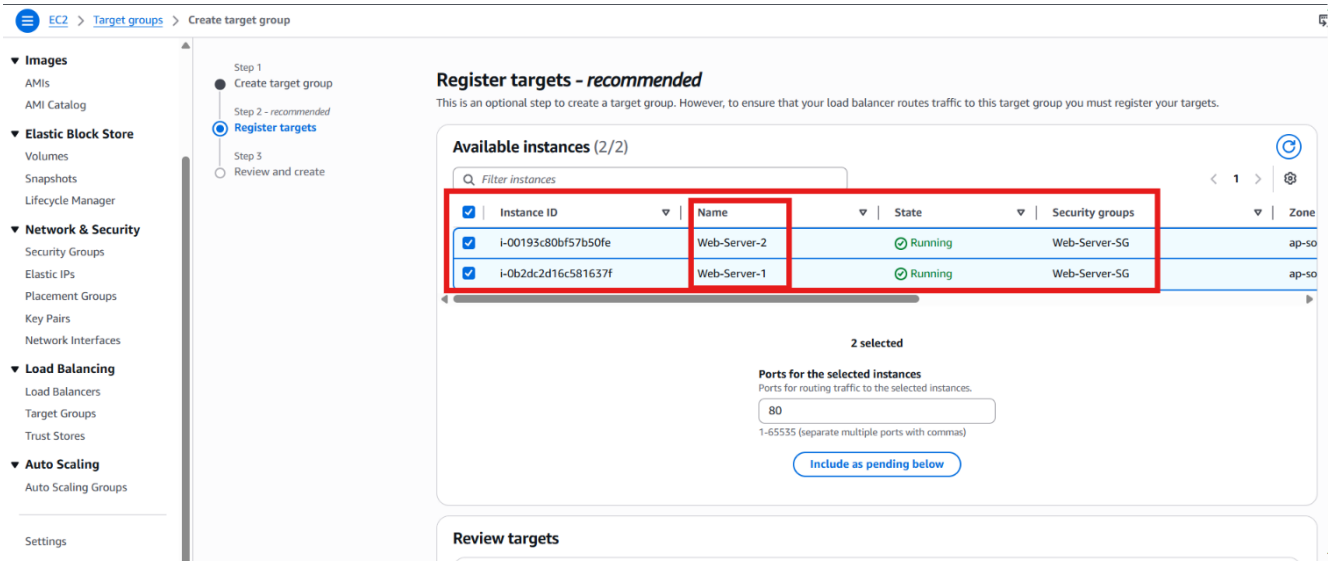
- Launch **Web-Server-1** using **Amazon Linux**, **t3.micro** and do the same for other servers.



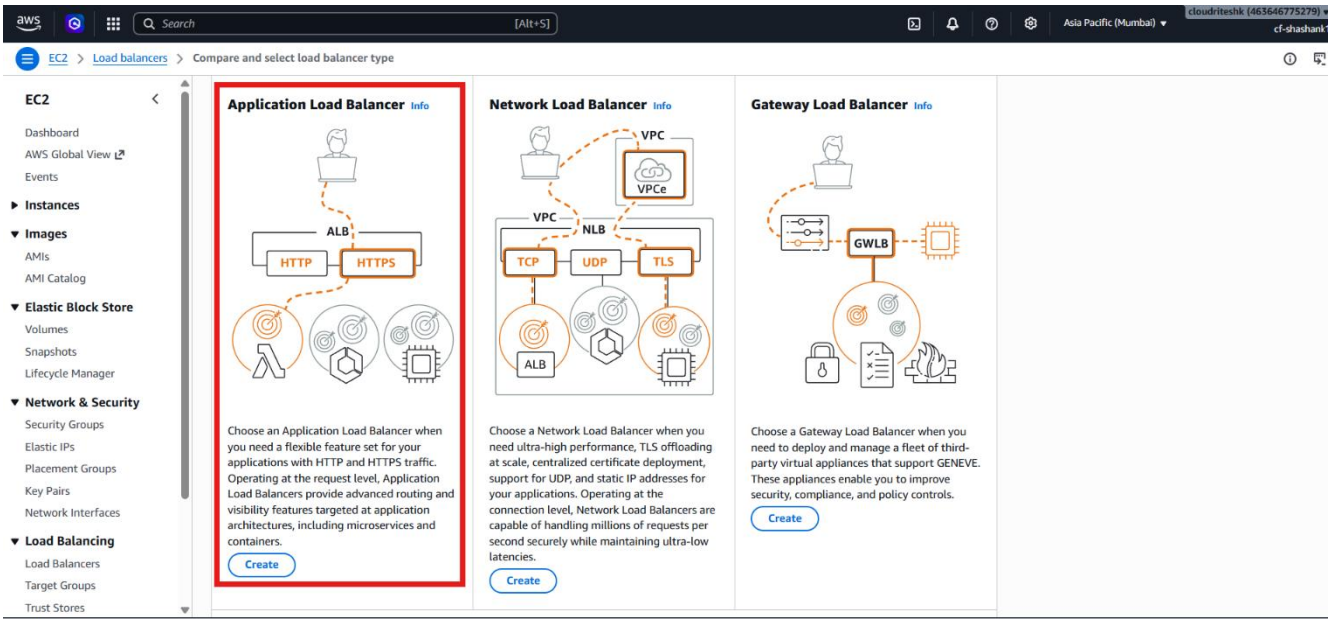
- Repeat the same two EC2 launches in the US region.
- Navigate to **Target Groups** and create **TG-India** with HTTP port 80 in **default VPC**.



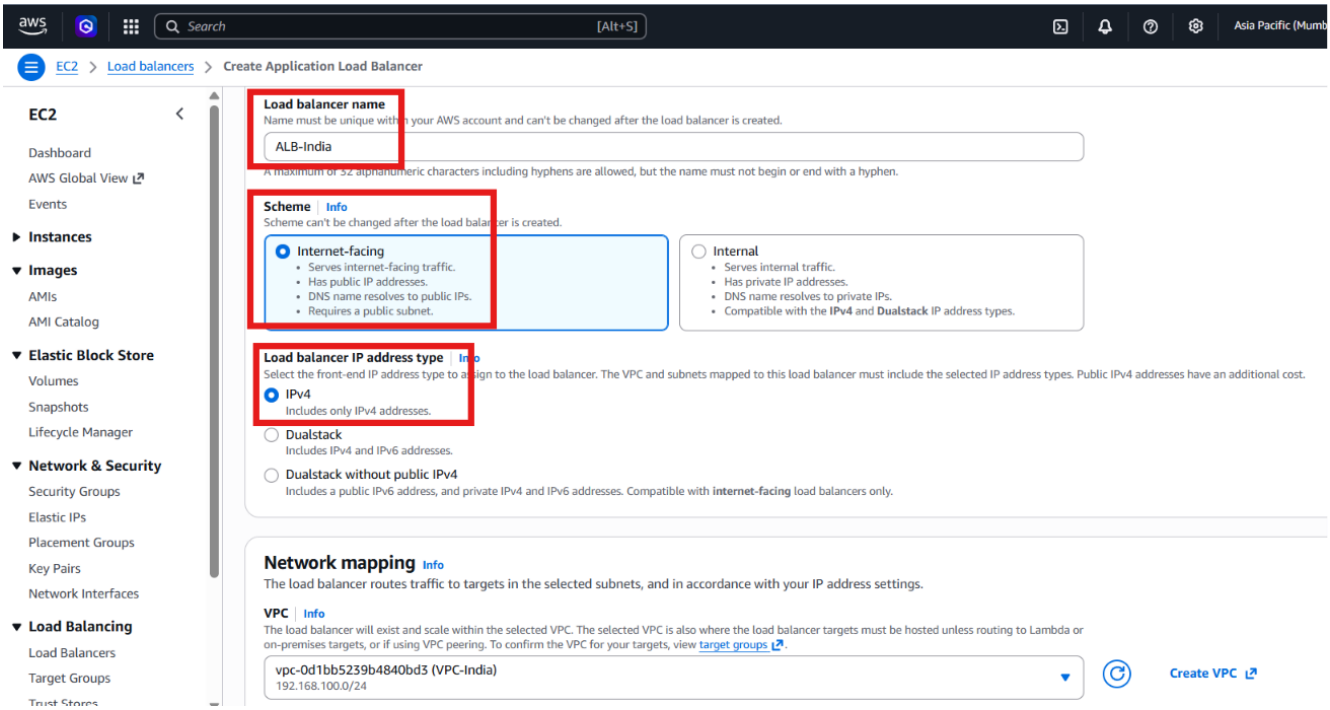
- Register both India EC2 instances as targets and click **Include as pending below** and click **Next.**

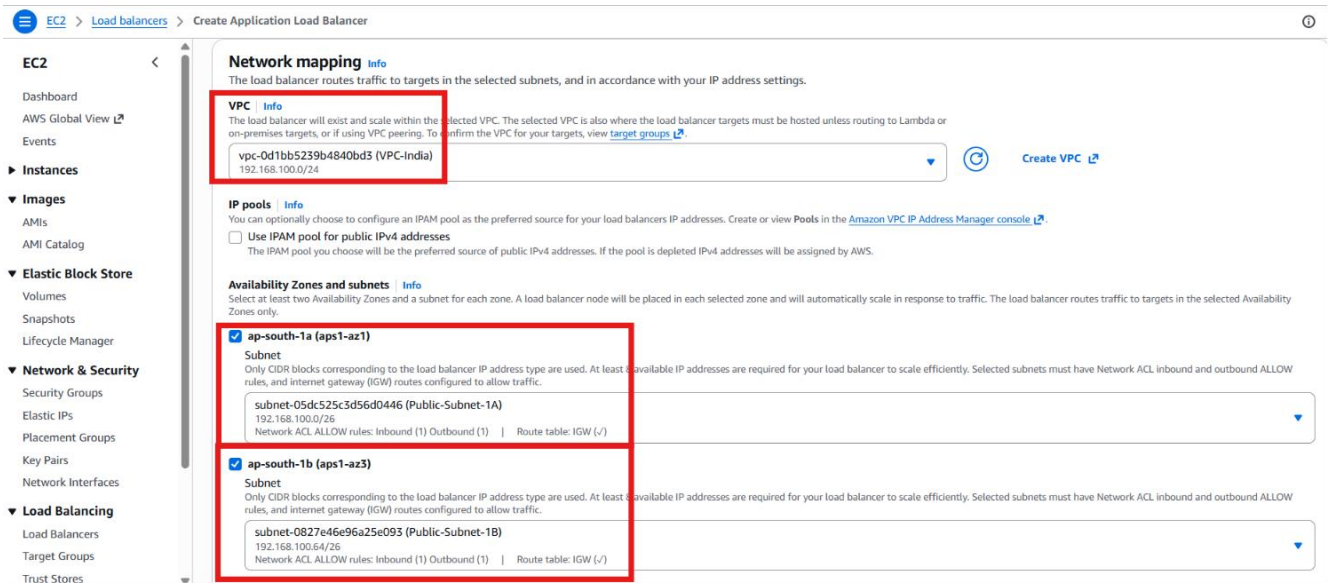


- Wait for the target health to become healthy.
- Now go to **Load Balancer** and choose **Application Load Balancer.**

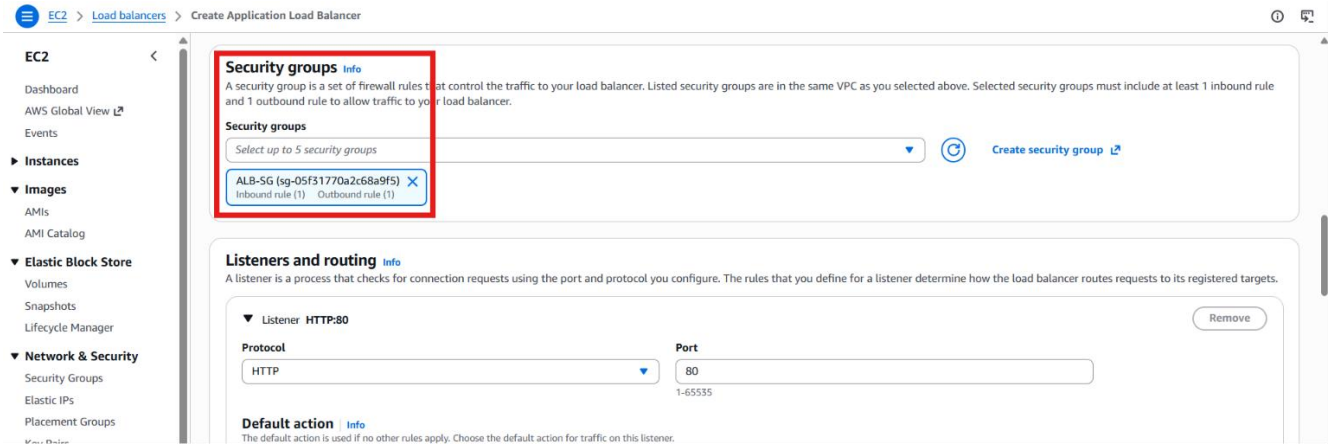


- Create an Application Load Balancer named **ALB-India** in the public subnets of **VPC-India** as shown below.

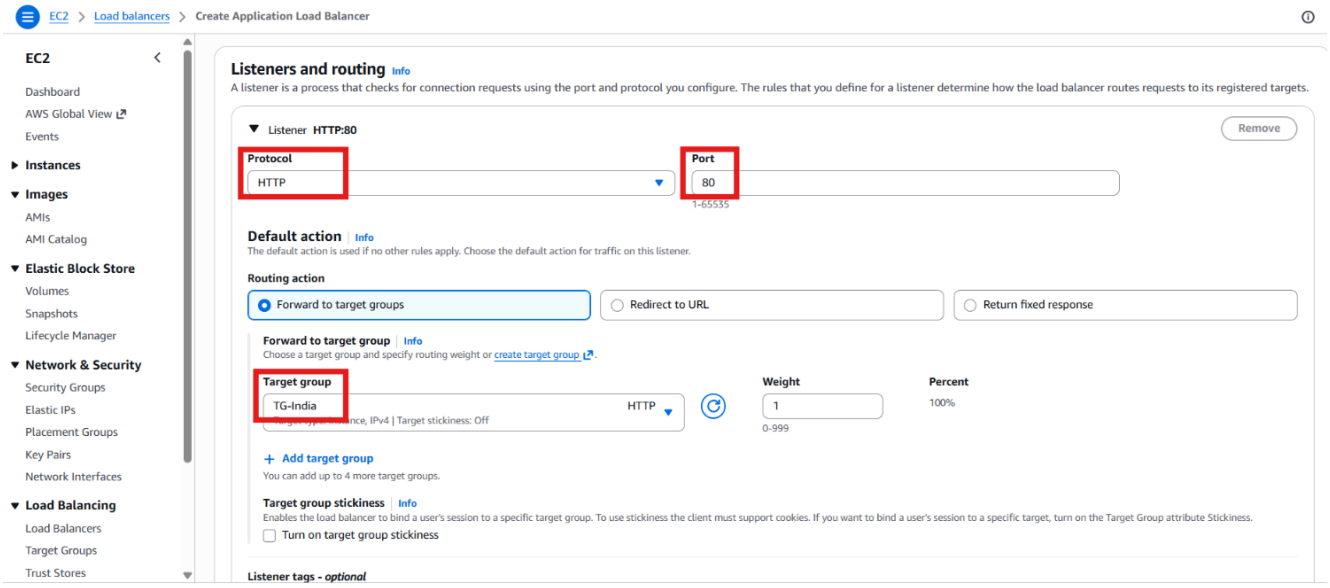




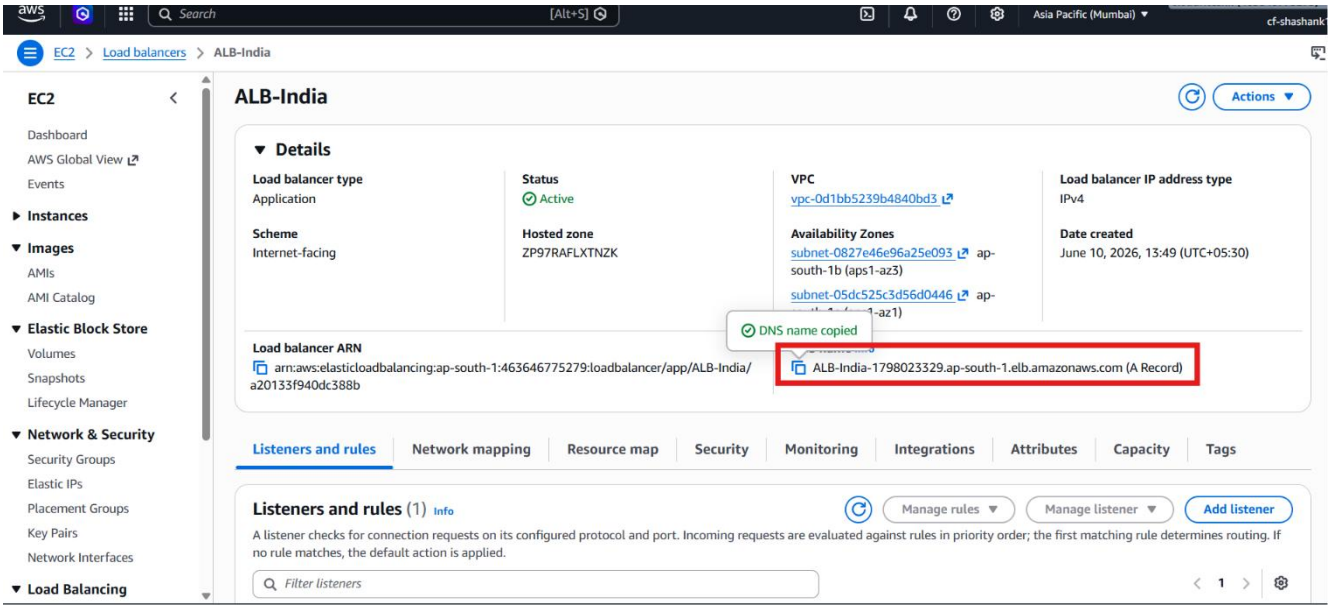
- Attach **ALB-SG** security group to the load balancer.



- Configure the listener on **HTTP 80** and forward traffic to **TG-India**.



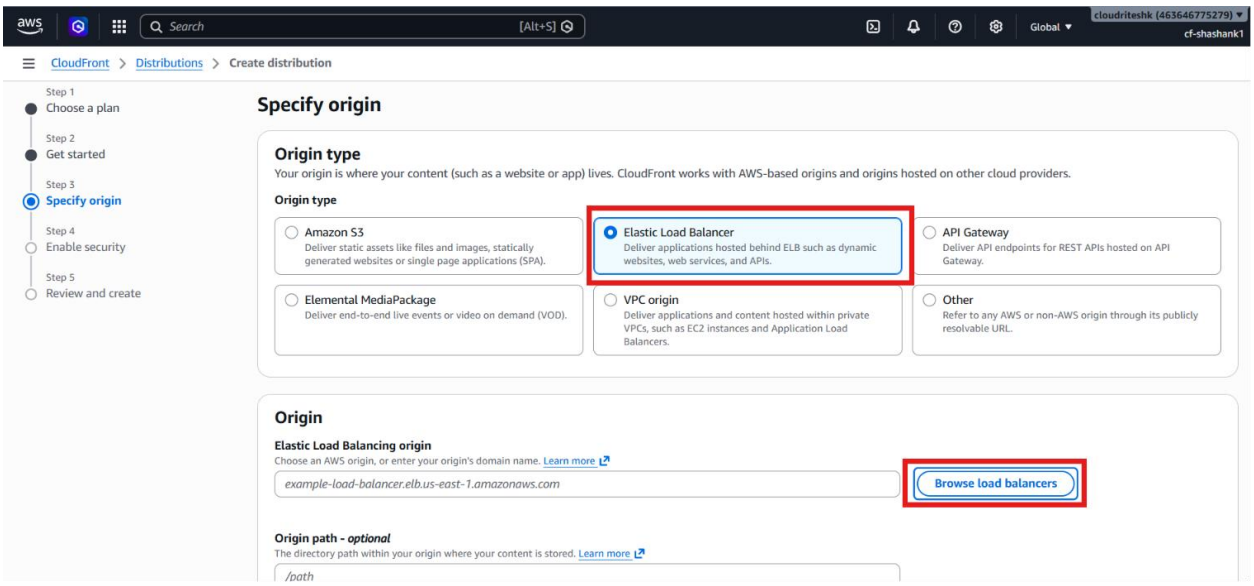
- Create the matching US target group and Application Load Balancer in **VPC-USA**.
- Verify the India ALB DNS name opens the website and alternates between the two India web servers.



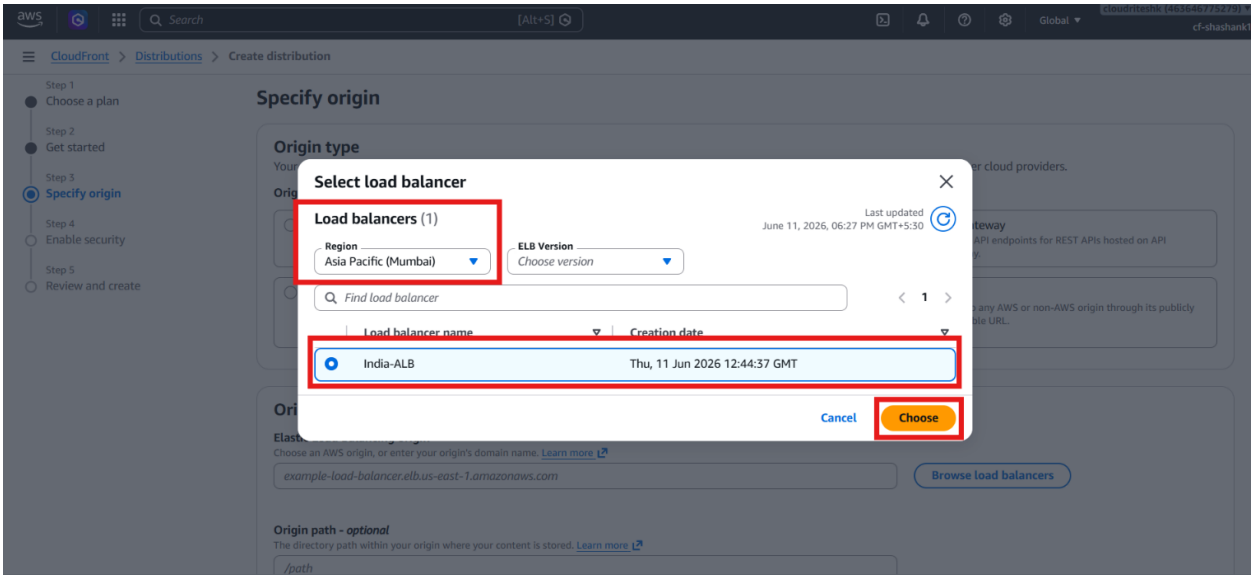
- Verify the US ALB DNS name opens the USA website and alternates between the two US web servers.

3. Create the CloudFront Distribution

- Navigate to **Amazon CloudFront**.
- Click **Create Distribution**.



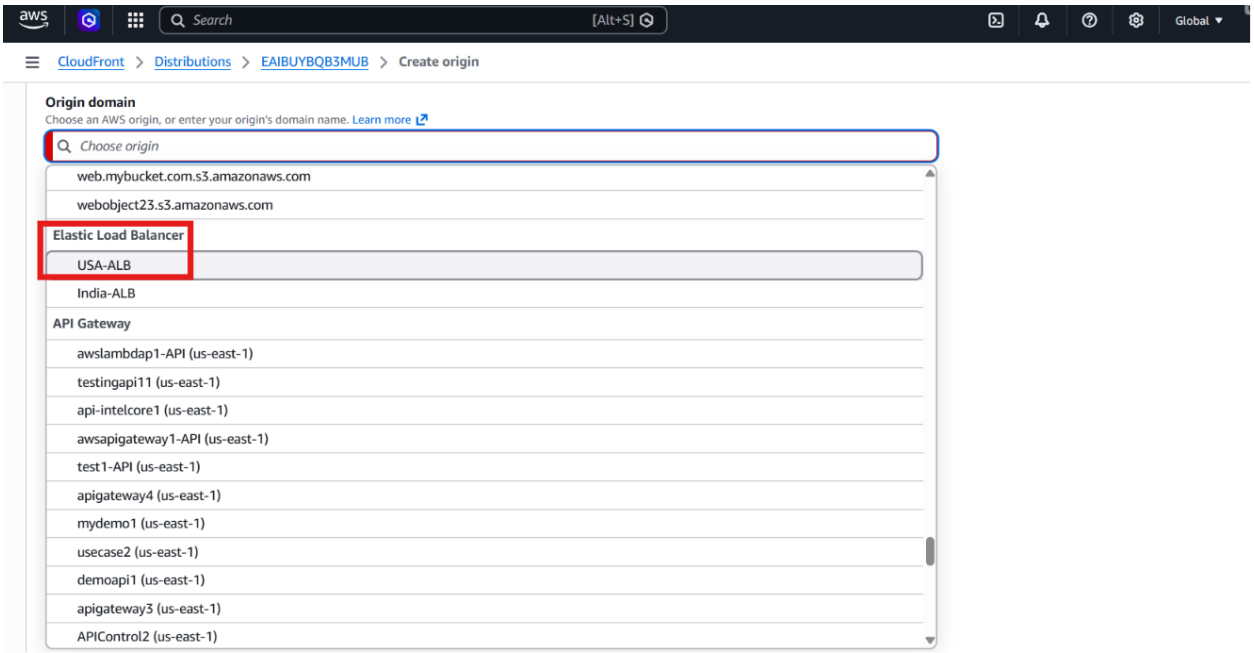
- Select the Mumbai load balancer as the origin.



- Configure **HTTP Only** communication.
- Set cache behavior according to lab requirements.
- Click **Create Distribution**.
- Wait for deployment.

4. Add the Backup Origin

- Open the CloudFront distribution.
- Navigate to **Origins**.
- Click **Create Origin**.

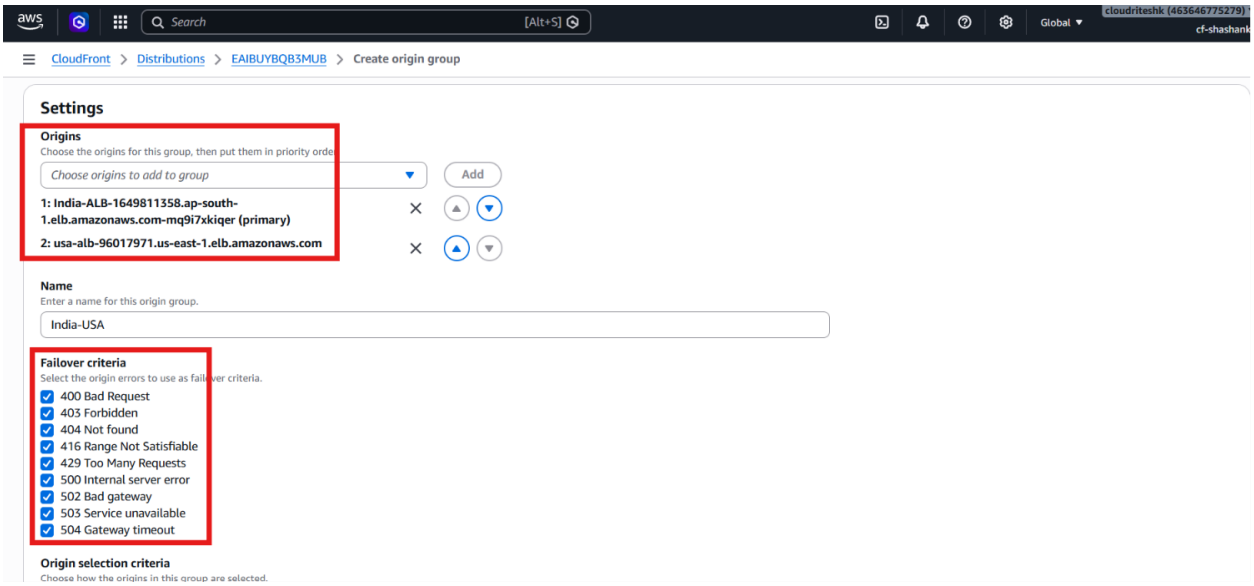


- Select the USA load balancer.
- Click **Create Origin**.

5. Create an Origin Group

Configuration Steps

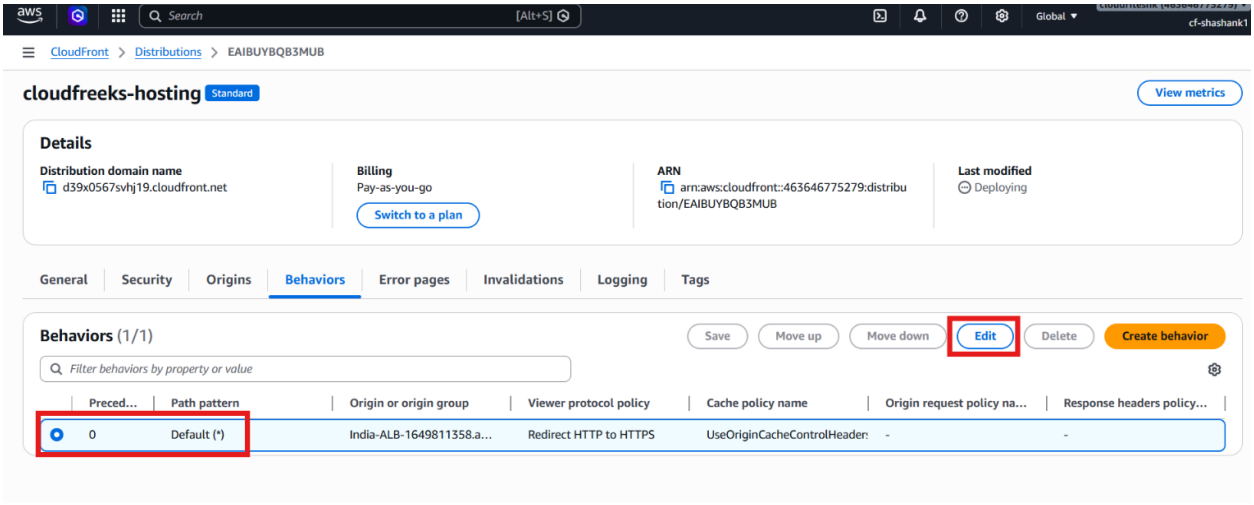
- Navigate to **Origins**.
- Click **Create Origin Group**.
- Select both load balancers.



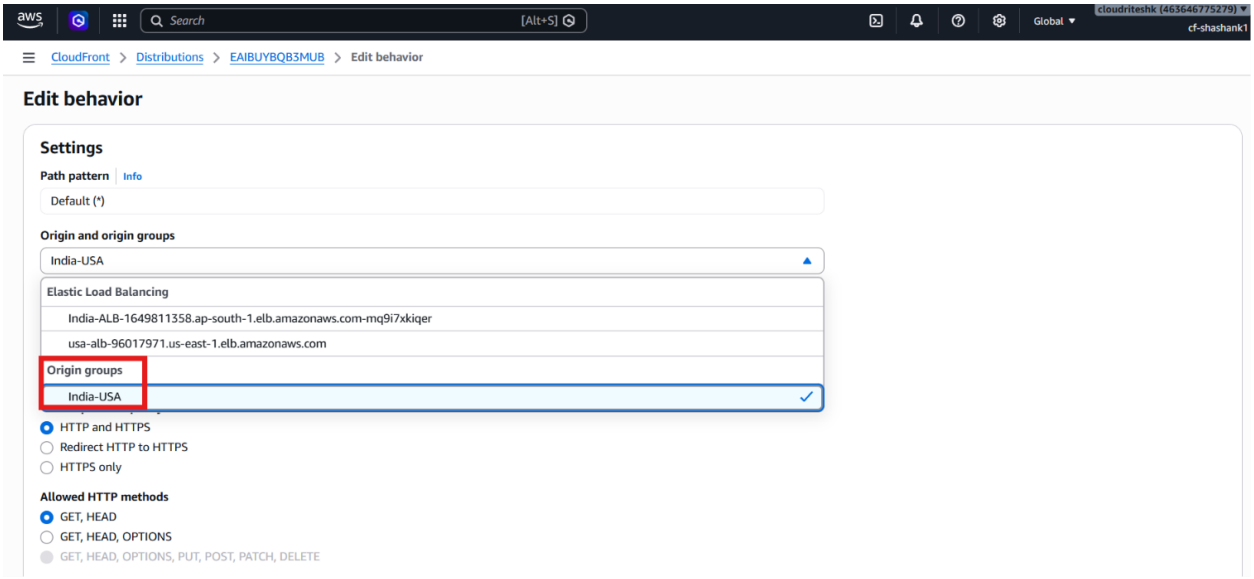
- Configure the Mumbai load balancer as **Primary Origin**.
- Configure the Virginia load balancer as **Secondary Origin**.
- Select the failover status codes.
- Click **Create Origin Group**.

6. Associate the Origin Group with CloudFront Behavior

- Navigate to **Behaviors**.
- Select the default behavior.
- Click **Edit**.



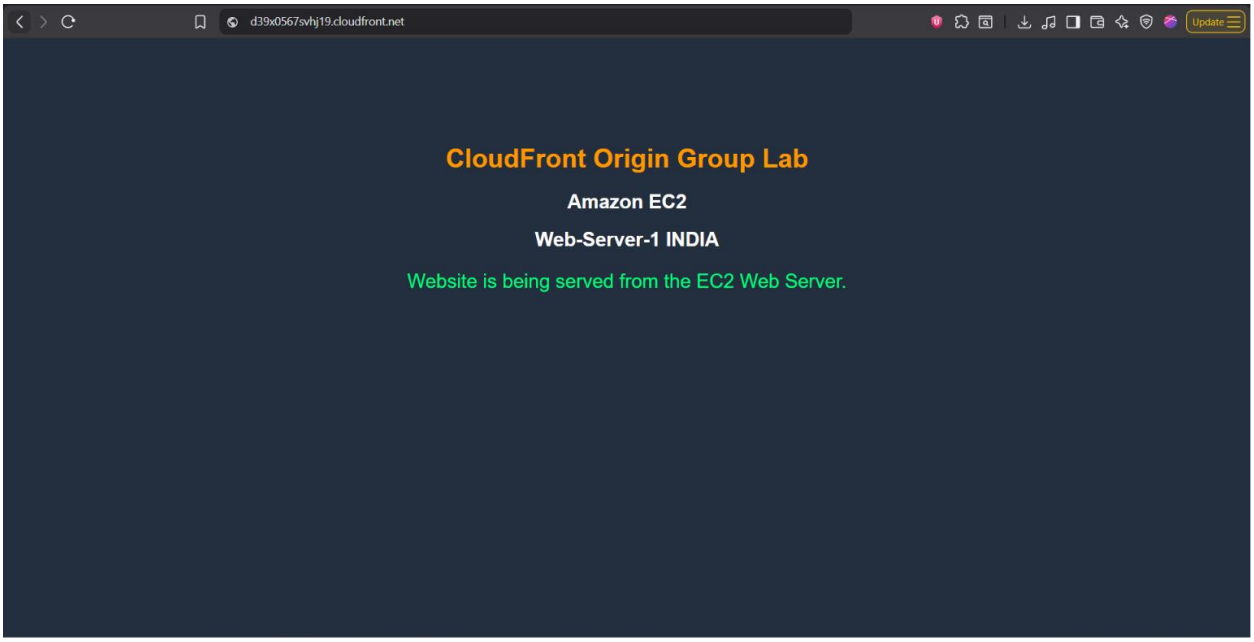
- Under **Origin and Origin Groups**, select the created origin group.



- Change **Cache Policy** to **CacheDisabled** only for lab purpose.
- Click **Save Changes**.

7. Verify Normal Traffic Flow

- Open the CloudFront domain name.
- Verify the India website loads successfully.



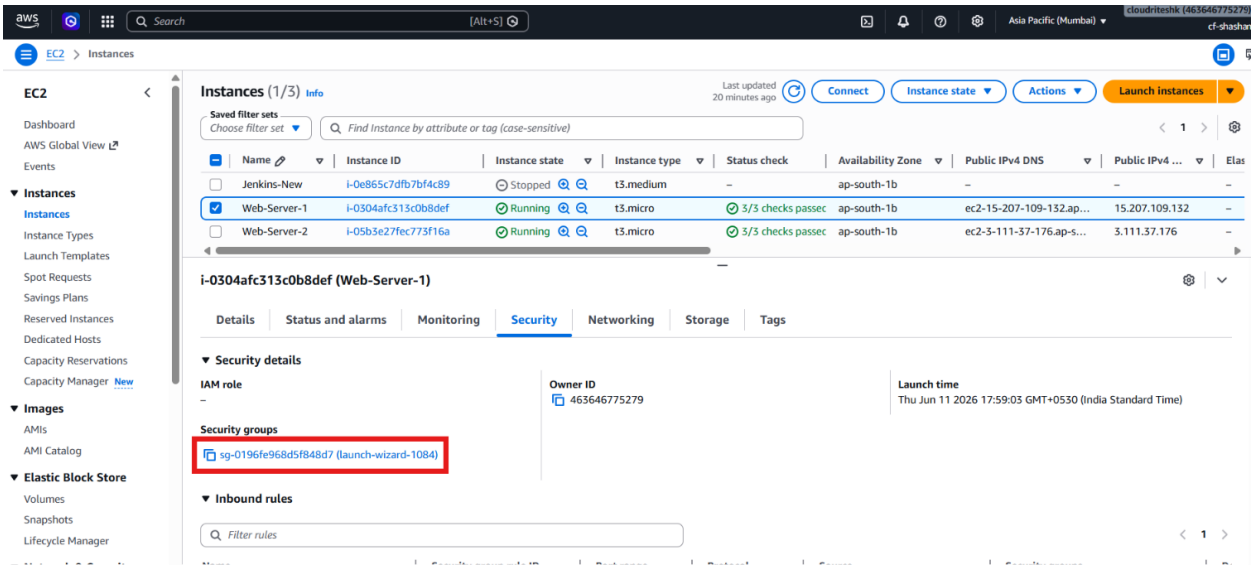
- Confirm requests are being served through the Mumbai load balancer.

8. Test Regional Failover

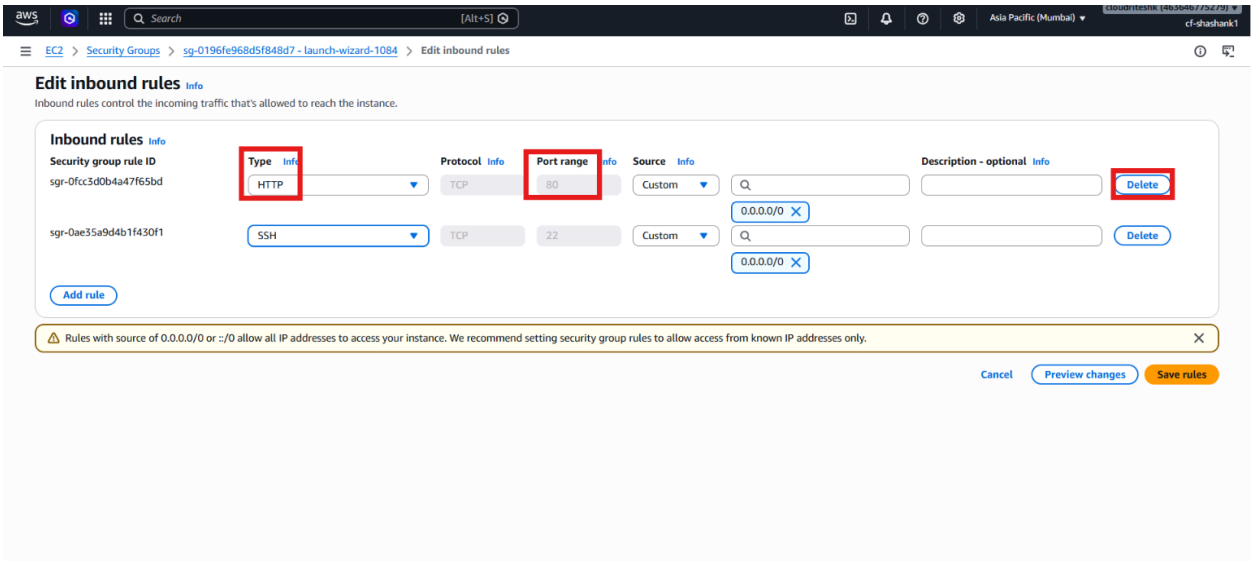
Failover Steps

- Navigate to **EC2 → Load Balancers**.

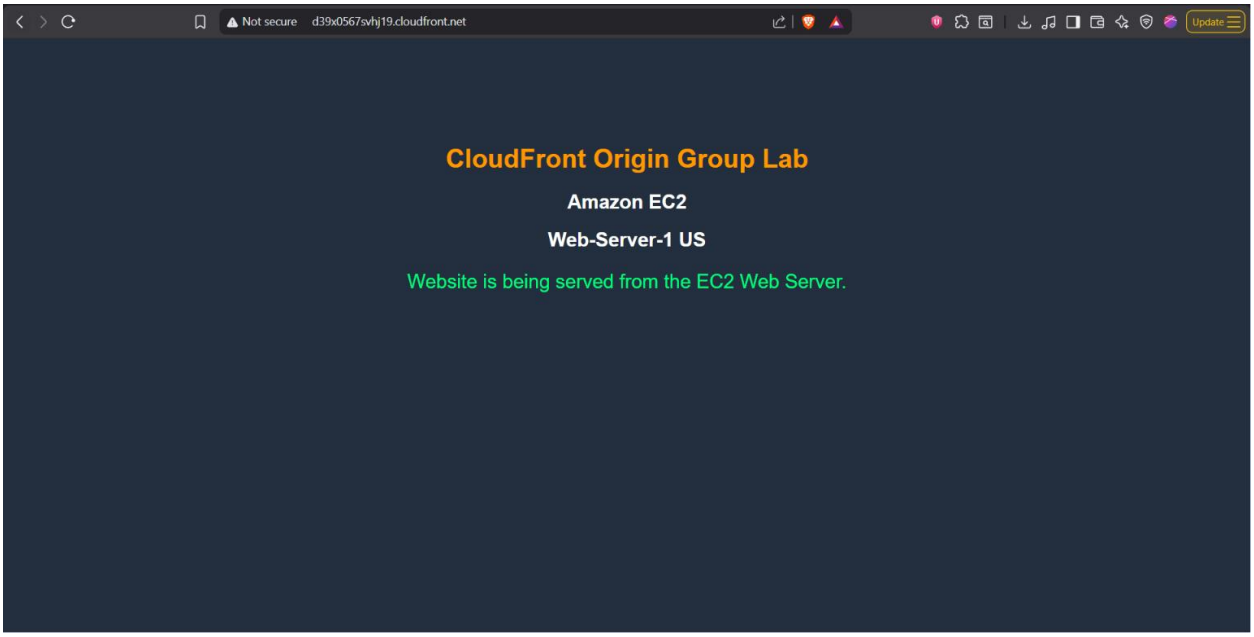
- Open the Mumbai load balancer.
- Locate the attached security group.



- Open **Inbound Rules**.
- Remove the HTTP port **80** rule.
- Save the changes.

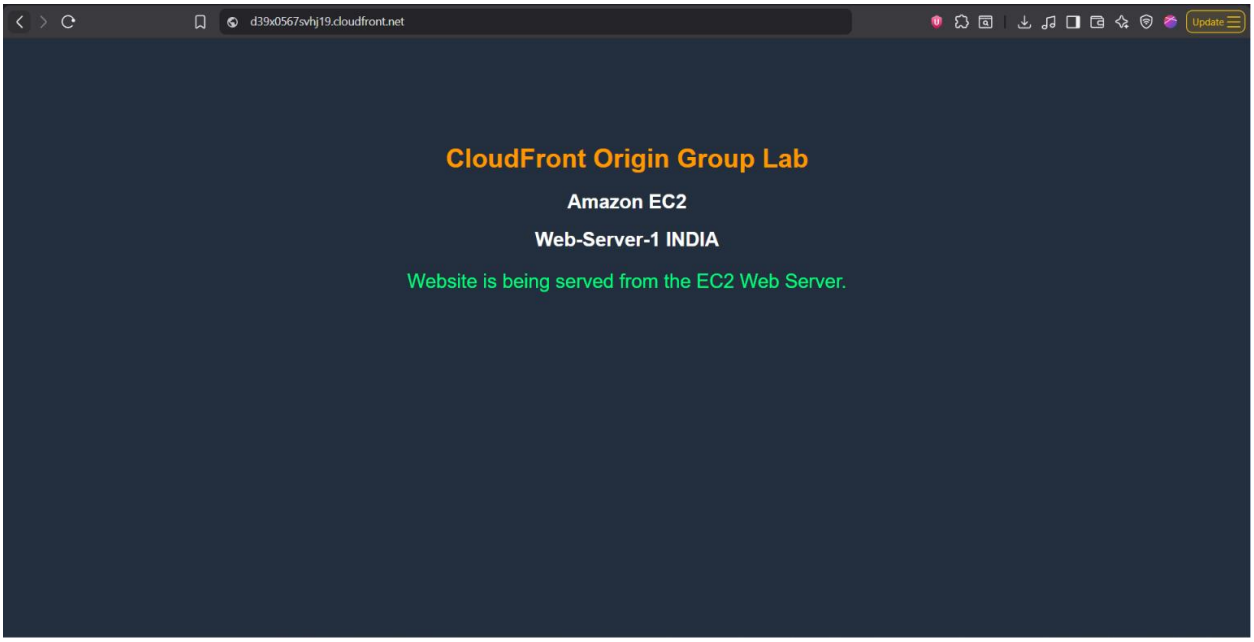


- Wait for CloudFront health checks.
- Refresh the CloudFront URL.
- Verify the US website appears.



9. Restore the Primary Region

- Open the Mumbai load balancer security group.
- Click **Edit Inbound Rules**.
- Re-add HTTP port **80**.
- Save the changes.
- Wait for health checks to succeed.
- Refresh the CloudFront URL.



- Verify traffic returns to the India website.

Verification

- The Mumbai Application Load Balancer serves as the primary origin.
- The Virginia Application Load Balancer serves as the secondary origin.
- CloudFront Origin Group is configured successfully.
- Traffic automatically fails over during primary origin failure.
- Traffic returns to the primary origin after recovery.
- Multi-region application availability is successfully achieved using **Amazon CloudFront Origin Groups**.